

PART A — User Stories & On-Chain Requirements Document

A.1 — Core User Personas (Final, Prioritized)

After brainstorming the full cast of people who touch FLUSH (see appendix B.1 for the long list), I narrowed to the **four** personas that matter for a Proof-of-Concept. The test I used: *does proving this persona's core loop prove the value proposition?* If not, the persona is real but can wait.

Persona 1 — The Crypto-Native Player (primary)

- **Who:** Holds USDC in a self-custodial Solana wallet (Phantom/Backpack/Solflare). Comfortable signing transactions. Has played online poker or wants to.
- **Core need:** Sit down at a real-stakes table, play a hand, and have winnings land back in their own wallet — without ever handing custody to a "house."
- **Why they're POC-critical:** They are the value proposition incarnate. If this loop doesn't work for them, nothing else matters.

Persona 2 — The Practicing Newcomer (primary)

- **Who:** Owns a wallet but is nervous about staking real USDC, or is still learning the format. The "curious wallet holder" at the top of the funnel.
- **Core need:** Play against AI at the same stake structure they'll eventually play for real, build confidence, then take the first real buy-in.
- **Why they're POC-critical:** They prove the single riskiest hypothesis in the whole product — that AI practice converts to staked play. The whole "onboarding friction" pillar rides on this persona.

Persona 3 — The Table Host / Game (system actor, but POC-critical)

- **Who:** Not a human — the on-chain program logic that opens a room, seats players, runs the betting rounds, escrows the pot, and pays the winner.
- **Core need:** Take buy-ins into escrow, enforce the rules of a hand, and pay out the correct amount to the correct wallet, trustlessly.
- **Why they're POC-critical:** This is the program. Every human story above resolves into *this* actor doing its job correctly. I'm listing it explicitly as a persona because in an on-chain product the "house" is code, and treating it as a first-class user keeps me honest about what the program must guarantee.

Persona 4 — The Operator / Admin (me, the developer)

- **Who:** Me. The deployer and maintainer of the program.
- **Core need:** Configure stake tiers, set/collect the rake, pause a table if something breaks, and not be able to touch players' stacks.

- **Why they're POC-critical:** A money-handling program with no admin controls is reckless; one where the admin *can* touch player funds breaks the value prop. The POC has to demonstrate the narrow, safe slice of admin power.

Personas deliberately deferred (not in POC): high-stakes whales, streamers/creators, leaderboard-status casual players, and ecosystem partners. All real, all in FINAL.md's market segments — but none of them are needed to *prove the core value*. They're acquisition and scale concerns, not proof-of-concept concerns.

A.2 — Function Map (Final)

Key functions each prioritized persona must be able to perform.

Crypto-Native Player

- Connect a Solana wallet to the app.
- Browse available stake tiers and see which rooms have open seats.
- Buy into a room at a chosen tier (stake USDC into escrow).
- Take a seat at a 6-player table auto-matched from the room.
- Act on their turn within the 8-second clock (check / call / raise / fold).
- Win a hand and have the pot settle to their wallet.
- Leave the table and withdraw their remaining stack to their wallet.

Practicing Newcomer

- Connect a wallet (no funds required to start).
- Enter an AI-staffed practice room at any tier with no USDC at risk.
- Play full hands against AI opponents at real stake structure.
- See live UX assists (current hand, best-hand probability, win %).
- Convert: take their first real buy-in at the tier they practiced.

Table Host / Game (program)

- Open a room for a given tier with its buy-in amount and seat cap.
- Accept and escrow a player's buy-in.
- Seat 6 players into a table from the room pool.
- Advance the hand through betting rounds, enforcing turn order and the timer.
- Determine the winner and compute the payout (pot minus rake).
- Release the pot to the winning wallet and return remaining stacks on leave.

Operator / Admin

- Initialize the program's global config (tiers, rake %, fee destination).
 - Create/enable a stake tier.
 - Collect accumulated rake to the fee destination.
 - Pause/resume a table in an emergency — **without** the ability to seize player stacks.
-

A.3 — Final User Stories (Refined through Parts B & C)

These are the de-jargoned, atomic, non-overlapping stories that survived the granularity pass. They are written so a non-technical stakeholder reads each one and knows exactly what one concrete thing happens.

Player onboarding & funds

- **US-1.** A player connects their own crypto wallet to FLUSH.
- **US-2.** A player chooses a stake level and puts their entry money (in USDC) into the table's locked holding spot.

Playing a hand

- **US-3.** A player is given a seat at a table once enough players are ready.
- **US-4.** A player makes one betting decision (bet, match, raise, or quit the hand) during their turn.
- **US-5.** When a player's turn timer runs out, the system automatically quits the hand for them.

Settling

- **US-6.** The winning player automatically receives the pot money to their own wallet.
- **US-7.** A player takes their remaining money out of the table back to their own wallet.

Practice & conversion

- **US-8.** A player enters a practice table staffed by computer opponents without putting any real money at risk.
- **US-9.** A player who has been practicing puts in their first real entry money at the same stake level.

Operator

- **US-10.** The operator sets up a new stake level that players can join.
- **US-11.** The operator collects the platform's small per-pot fee into the platform's account.
- **US-12.** The operator temporarily freezes a table in an emergency, and freezing never lets the operator take any player's money.

Atomicity note: the original "player signs up and buys in" was split into US-1 and US-2. "Win and cash out" was split into US-6 (pot settles on win) and US-7 (withdraw

remaining stack), because winning the pot and leaving the table are two distinct on-chain actions at two different times. Full log in C.1.

A.4 — Core POC Requirements (Final)

The two most critical interaction paths — the ones that, if they work, prove the product — are:

1. **The real-money loop:** buy in (US-2) → get seated (US-3) → act on turn (US-4) → win settles to wallet (US-6) → withdraw (US-7).
2. **The conversion loop:** practice against AI (US-8) → take first real buy-in (US-9).

Technical requirements derived from those paths:

- **Wallet integration** — connect, read USDC balance, request signatures for buy-in/withdraw transactions.
 - **A USDC escrow account per game**, owned by a Program-Derived Address (PDA) so no human key controls the pot.
 - **A room/table state account** holding tier, buy-in size, seat list, pot total, whose-turn, and a turn deadline (slot/timestamp).
 - **An on-chain settlement instruction** that computes winner payout = pot – rake and transfers USDC to the winner.
 - **A withdraw instruction** that returns a leaving player's remaining stack from escrow to their wallet.
 - **A randomness/shuffle boundary** — for POC, a commit-reveal or VRF-backed deck seed so deal order isn't manipulable. (Flagged as the highest-risk subsystem; see FINAL.md gaps.)
 - **An off-chain game server** running the AI opponents and the 8-second clock, with the practice mode entirely off-chain (no escrow) so US-8 risks nothing.
 - **A clean off-chain → on-chain handoff** for US-9, so "practice" and "real" use the same UI and the only difference is whether USDC is escrowed.
-

PART B — Process Appendix

B.1 — Manual User Brainstorming (Part A, Step 1)

Direct users (day-to-day):

- Crypto-native poker players (the core)
- Online poker grinders coming from PokerStars/GG
- Casual mobile poker players / play-money refugees

- Complete poker newcomers using AI mode
- High-roller crypto whales (\$5K/\$10K tier)

Indirect users / beneficiaries:

- Streamer audiences who play alongside a creator
- Friends invited via referral links
- Wallet providers (Phantom/Backpack) who benefit from on-chain activity

Administrators / moderators:

- Me, the developer/operator (deploy, config, rake, pause)
- A future community/Discord moderator (anti-collusion reports)
- A future support person handling "my tx failed" tickets

Stakeholders (vested but maybe don't play):

- Investors / future token-or-equity holders
- Solana ecosystem partners (co-marketing)
- A security auditor (has to vouch for the escrow contract)
- Regulators / legal (the uncomfortable stakeholder I can't ignore)

System "actors" (not human, but they act):

- The poker engine / game server (deals cards, runs the clock)
 - The on-chain program itself (escrow + payout logic)
 - The AI bot opponents in practice rooms
-

B.2 — AI-Assisted User Prioritization (Part A, Step 2)

Prompt I sent

"My project's value proposition is: instant, trustless, stake-based poker on Solana where chips are real USDC, the seat settles on-chain, and the table is always ready — collapsing custody risk, liquidity drag, and the play-money dead-end. Here is a brainstormed list of all potential user types: [pasted the entire B.1 list]. Based on the value proposition, which 2–5 of these user types are the most critical to focus on for an initial Proof-of-Concept? For each, give a brief rationale for why they're essential to proving the core value."

AI output (summarized)

The AI recommended four:

1. **Crypto-native poker player** — "the embodiment of your value prop; the real-money on-chain loop must work for them first."

2. **The newcomer in AI practice mode** — "the only way to test your conversion hypothesis, which is your stated riskiest assumption."
3. **You, the operator/admin** — "any money-handling system needs configuration and emergency controls; without this the POC is incomplete."
4. **The on-chain program / escrow logic** — the AI initially folded this into "infrastructure, not a user," but when I re-read its answer it kept describing what "the system must guarantee," so it was implicitly treating it as an actor.

Where I agreed / disagreed

- **Agreed** on players 1 and 2 immediately — these were always going to be primary.
- **Agreed** on deferring whales/streamers/casual. That matches the sequencing note I already wrote in FINAL.md (don't spend on casual UA until high tiers are populated). It would've been inconsistent to make them POC-critical here.
- **Disagreed / promoted:** the AI was wishy-washy about the **on-chain program as a persona**. I made a deliberate call to elevate it to a first-class persona (Persona 3). In a custodial product you'd never list "the database" as a user, but here the program *is* the house, and the entire trust story is "you're trusting code, not a company." If I don't treat the program as an actor with explicit obligations, I'll under-specify what it must guarantee. So I overrode the AI's "it's just infrastructure" framing.
- **Kept the operator (me)** despite a small temptation to cut it — a money program with no admin path is irresponsible, and more importantly the *constraint* on admin power (can't seize stacks) is itself a value-prop-defining feature worth proving.

Final prioritized list: Crypto-Native Player, Practicing Newcomer, Table Host/Game (program), Operator/Admin. (4 personas — within the 2–5 band.)

B.3 — Core Function Mapping (Part A, Step 3)

Prompt I sent

"For a project with this value proposition [pasted], focusing on these prioritized users — crypto-native player, practicing newcomer, the on-chain game program itself, and the operator/admin — help map the key functions or interactions each user needs to perform."

AI output (summarized) + my edits

The AI gave a solid first cut. A few notes on what I changed:

- It listed "deposit funds to account balance" for the player. I **cut that** — there is no account balance in FLUSH; that's the custodial model we're explicitly killing. I rewrote it as "buy into a room (stake USDC into escrow)" and "withdraw remaining stack," which keeps custody with the player between hands.
- It missed the **8-second turn clock** as a function entirely. I added "act within the clock" and

the auto-fold-on-timeout behavior, because the time-box is a named product feature, not an implementation detail.

- For the program, it gave generic "manage game state." I broke that into the concrete obligations (escrow buy-in, seat 6, advance betting rounds, compute payout minus rake, release pot, return stacks).
- For the operator it suggested broad admin powers; I narrowed it and explicitly wrote in the **negative constraint** (pause cannot seize stacks). The AI doesn't think to specify what an actor *can't* do — that's on me.

Result is the function map in A.2.

B.4 — Deriving Core POC Requirements (Part A, Step 4)

My manual pick of the 1–2 critical paths

1. The real-money loop: buy in → seated → act → win settles → withdraw.
2. The conversion loop: practice vs AI → first real buy-in.

I picked these because path 1 is the value proposition's "custody + liquidity" pillars and path 2 is the "onboarding" pillar plus the riskiest hypothesis (does AI practice actually convert?).

Prompt I sent

"Based on these two critical interactions — (1) a player buys into an escrowed table with USDC, plays a timed hand, and the pot settles directly to the winner's wallet, then they withdraw their remaining stack; and (2) a player practices against AI with zero USDC at risk, then takes their first real buy-in at the same stake — what are the key technical requirements to build a proof-of-concept?"

AI output (summarized)

It returned: wallet integration, a PDA-owned escrow per game, a game/table state account, a settlement instruction (payout = pot – rake), a withdraw instruction, a randomness source for the shuffle, an off-chain game server for the clock + AI, and a clean handoff between practice (off-chain) and real (on-chain).

My edits

- The AI initially put practice mode partly on-chain. I **forced it fully off-chain** — if practice touches escrow at all, US-8's "no real money at risk" promise is a lie and we'd be paying compute for fake games. The only on-chain moment in the conversion loop is US-9's first real buy-in.
- I bumped **randomness/shuffle** to "highest-risk subsystem" to stay consistent with the honest-gaps section of FINAL.md (commit-reveal/VRF is new ground for me).

Result is A.4.

Part B (of the assignment) — Adversarial Analysis & Granularity Check

Prompt I sent

"Review my core user functions/stories [pasted A.3 draft] and requirements [pasted A.4]. Considering my value proposition — non-custodial, USDC-denominated, on-chain-settled poker with AI practice onboarding — do these stories truly hit the mark? Are the requirements granular enough to map to specific technical components (database schemas, API endpoints, blockchain programs/accounts)? What's missing or unclear?"

AI critique (summarized — the points that actually landed)

1. **"Player buys in and gets seated" is two stories crammed together.** Buying into escrow and being auto-matched to a seat are separate events at separate times. Split them.
2. **"Win and cash out" conflates two on-chain actions.** Winning the pot (settlement, automatic, at hand end) and withdrawing your remaining stack (manual, when you leave) are different instructions touching escrow differently. Split them.
3. **The timeout behavior was implicit.** "Act on your turn" doesn't say what happens when the clock hits zero. That's a distinct, must-specify rule (auto-fold). Make it its own story.
4. **Jargon leak.** Several stories said "stake USDC into the escrow PDA" — a non-technical stakeholder doesn't know what a PDA is. De-jargon the *story* layer; keep the jargon in the requirements layer.
5. **Admin powers under-specified on the safety side.** "Operator can pause a table" — can a paused table's funds be touched? If you don't say no, a reader assumes maybe. Make the constraint explicit in the story.
6. **Requirements granularity is decent but the randomness requirement is a hand-wave.** "A randomness source" isn't granular — name the mechanism (commit-reveal vs VRF) and acknowledge it's the risk.
7. **Missing story: practice → real is described as a funnel but you never wrote the actual conversion as a user action.** Add an explicit "takes first real buy-in" story so the hypothesis is testable as a concrete event.

My analysis & what I did

All seven are fair. Points 1, 2, and 7 are the ones I'd have shipped wrong without the nudge — I have a habit of writing "and then" stories that smuggle two actions into one. Points 4 and 5 are the kind of thing that reads fine to me (I know what a PDA is) but would confuse the non-technical stakeholder this document is partly for.

The only place I held a slightly different line: on point 6, I kept the randomness mechanism *named but undecided* (commit-reveal OR VRF) rather than committing, because honestly I haven't built either yet and pretending I've decided would be the same self-flattery I called out in FINAL.md. So I documented it as the highest-risk open subsystem instead of faking certainty.

Every other point I applied directly. The refinements flow into Part C below.

Part C — Granularity & Clarity Refinement Log (C.1)

I went through every story with the de-jargon / atomic / single-action / no-overlap lens. Here's the change log.

#	Before	After	Rationale
C-1	"A player signs up and buys into a table."	US-1 "connects their wallet" + US-2 "puts entry money into the table's locked holding spot."	Atomicity — connecting a wallet and staking funds are two separate actions at two times.
C-2	"A player buys in and is seated at a table."	US-2 (buy-in) stays; US-3 "is given a seat once enough players are ready" split out.	Atomicity — buy-in escrows funds; seating happens later when the room fills to a table. Different events.
C-3	"A player acts on their turn (check/call/raise/fold), and folds automatically if time runs out."	US-4 "makes one betting decision during their turn" + US-5 "system auto-quits the hand when the timer runs out."	Atomicity — the active decision and the passive timeout are distinct; the timeout is a system action, not a player action.
C-4	"A player wins and cashes out."	US-6 "winning player automatically receives the pot" + US-7 "player takes remaining money out back to their wallet."	Atomicity — pot settlement is automatic at hand end; withdrawal is a separate manual action when leaving.
C-5	"Player stakes USDC into the escrow PDA."	US-2 "puts their entry money (in USDC) into the table's locked holding spot."	De-jargon — replaced "escrow PDA" with "locked holding spot" so a non-technical stakeholder understands; PDA stays in the requirements layer (A.4).
C-6	"Player joins an AI room."	US-8 "enters a practice table staffed by computer opponents without putting any real money at risk."	Clarity of action + de-jargon — made the zero-risk promise explicit and replaced "AI room" with plain language.
C-7	(missing)	US-9 "a player who has been practicing puts in their first real entry	Missing story — the conversion event was described as a funnel

#	Before	After	Rationale
		money at the same stake level."	but never written as a concrete action; added so the core hypothesis is a testable event.
C-8	"Operator can pause a table."	US-12 "operator temporarily freezes a table in an emergency, and freezing never lets the operator take any player's money."	Clarity / no-overlap of power — added the explicit safety constraint so the story can't be read as "admin can touch funds."
C-9	Two near-duplicate stories: "player withdraws winnings" and "player cashes out stack."	Merged into US-7 (withdraw remaining stack). Winnings already land via US-6 settlement.	No overlap — winnings arriving (US-6) and withdrawing your stack (US-7) were being described twice; merged the redundant withdrawal.
C-10	"Admin manages tiers."	US-10 "operator sets up a new stake level." + US-11 "operator collects the platform's small per-pot fee."	Atomicity — "manage" hid two separate actions (create a tier, collect rake); split them.

Part D — Potential On-Chain Requirements per User Story

For each final story from Part C, the bulleted on-chain requirements. This is the translation layer from "what the user does" to "what the program/accounts must do."

US-1 — Player connects their wallet.

- No on-chain state change; this is a client-side wallet connection.
- Read the player's USDC associated-token-account balance for display.

US-2 — Player puts entry money (USDC) into the table's locked holding spot.

- Need a per-game **escrow token account owned by a PDA** (no human signer controls it).
- Need an instruction that transfers the tier's buy-in amount of USDC from the player's wallet into escrow.
- Must record the player's wallet address and escrowed amount on the table/room state account (for payout permissions and stack accounting).
- Must reject the transfer if the amount $\neq$ the tier's fixed buy-in, or if the room is full.

US-3 — Player is given a seat once enough players are ready.

- The room state account must track a list of bought-in players and open seats (cap 22 room / 6 per table).
- Need an instruction (or program logic) that assigns up to 6 players from the room pool into a table and initializes that table's hand state.
- Must initialize per-hand state: whose turn, current bet, pot = sum of blinds/antes, turn deadline.

US-4 — Player makes one betting decision during their turn.

- Need an instruction accepting an action enum (check/call/raise/fold) plus amount.
- Must verify the signer is the player whose turn it currently is.
- Must verify the action is legal given the current bet (e.g., raise $\geq$ min-raise) and the player's escrowed stack covers it.
- Must update pot, current bet, and advance the turn pointer + reset the turn deadline.

US-5 — System auto-quits the hand when the timer runs out.

- The table state must store a **turn deadline as an on-chain slot/timestamp**.
- Need an instruction (crankable by the game server or any caller) that, if `now > deadline`, folds the current player and advances the turn.
- Must be permissionless to call so no single party can stall the table.

US-6 — Winning player automatically receives the pot.

- Need a settlement instruction that determines the winning wallet (hand-rank evaluation result supplied/verified) and computes **payout = pot - rake**.
- Must transfer payout USDC from the escrow PDA to the winner's wallet.
- Must accrue the rake portion to a fee/treasury account.
- Must reset the hand state (pot $\rightarrow$ 0) and update each seat's remaining stack in escrow.

US-7 — Player takes their remaining money out to their wallet.

- Need a withdraw instruction that transfers a leaving player's remaining escrowed stack back to their wallet.
- Must verify the player is not in the middle of an active hand (no withdrawing mid-pot).
- Must zero out / remove the player's entry on the table state and free the seat.

US-8 — Player enters a practice table with no real money at risk.

- **No on-chain requirement** — practice runs entirely off-chain (no escrow, no USDC). Intentional, so the "zero risk" promise is structurally true.
- Off-chain only: game server seats the player with AI opponents and play-chips.

US-9 — Practicing player puts in their first real entry money at the same stake.

- Identical on-chain requirements to US-2 (escrow buy-in at the tier).

- Client requirement: the same UI path hands off from off-chain practice to the on-chain buy-in so the only difference the user feels is "now it's real."

US-10 — Operator sets up a new stake level.

- Need a global config account (PDA) storing the list of tiers, each with a fixed buy-in amount and rake %.
- Need an admin-gated instruction to add/enable a tier, signer-checked against the stored admin authority.

US-11 — Operator collects the platform's per-pot fee.

- Rake must accrue to a program-controlled fee/treasury account on each settlement (US-6).
- Need an admin-gated instruction to transfer accrued rake from the treasury to the configured fee destination.
- Must verify the caller is the stored admin authority.

US-12 — Operator freezes a table in an emergency, with no power to take player funds.

- The table state needs a **paused** flag; an admin-gated instruction toggles it.
 - While paused, betting instructions reject — but **escrow funds remain owned by the PDA and are only ever released to players via US-6 settlement or US-7 withdrawal.**
 - The program must contain **no instruction** that lets the admin transfer escrow funds to an admin-controlled account. (The safety guarantee is the *absence* of such an instruction, not a check inside one.)
-